



UNIVERSIDAD
NACIONAL
DE COLOMBIA



Sesión 6: Bucles, Reducciones y loop/recur

Semillero en Ciencias de la Computación

Santiago López Tomas Rodríguez

Junio 5, 2026



1. Recapitulando
2. La forma de un bucle
3. Profundizando en reduce
4. reductions: conservar la historia
5. Cuando reduce no alcanza: loop/recur
6. ¿Qué herramienta uso?
7. Ejercicios Prácticos

Idea de hoy

Cuando un bucle no transforma uno a uno (`map`) ni descarta elementos (`filter`), casi siempre está haciendo otra cosa: **acumular** un resultado (**reducción**).

Reconstruiremos bucles imperativos con `reduce`, agregamos `reductions` para conservar la historia del cálculo y dejamos `loop/recur` para el caso en que ninguna función de alto nivel encaje.



1. Recapitulando



- **map:** aplica una función a cada elemento y devuelve una colección del mismo tamaño.
- **filter:** conserva los elementos que cumplen un predicado.
- **reduce:** combina todos los elementos en un único resultado.

¿Podemos implementar *map* o *filter* usando *reduce*?



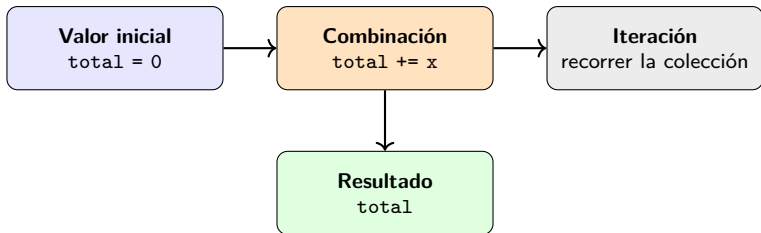
2. La forma de un bucle

Sumemos los movimientos de una cuenta en Java:

```
int total = 0;
for (int x : movimientos) {
    total += x;
}
return total;
```

- ¿Qué pasa si queremos el producto, el máximo o el conteo?
- Tendríamos que reescribir el bucle entero cada vez.

Todo bucle acumulador tiene la misma estructura:



Si separamos esas piezas, el bucle deja de ser un bloque rígido y se vuelve configurable.

Esas piezas son los argumentos de reduce:

```
(reduce combinacion valor-inicial coleccion)
```

```
(reduce + 0 movimientos)
```

- La **combinación** es una función (fn [acc x] ...).
- El **valor inicial** es el estado del acumulador antes de empezar.
- La **colección** es lo que se recorre, una sola vez.

Cambiar de cálculo ya no significa reescribir un bucle, sino cambiar dos argumentos:

```
(reduce + 0 xs)           ;; suma
(reduce * 1 xs)           ;; producto
(reduce max xs)           ;; maximo
(reduce (fn [n _] (inc n)) 0 xs) ;; conteo
```

El bucle imperativo mezclaba estas decisiones con la mecánica de recorrer.

Pregunta

¿Podemos implementar *map* o *filter* usando *reduce*?

Pista: El acumulador no necesariamente tiene que ser un número (Ejemplo: función identidad para colecciones).

map y filter también son reducciones

Si el acumulador es una colección que vamos construyendo con `conj`, podemos ver a *map* y *filter* como reducciones:

```
;; map f como reduce  
(reduce (fn [acc x] (conj acc (f x)))  
        [] xs)
```

```
;; filter pred como reduce  
(reduce (fn [acc x] (if (pred x) (conj acc x) acc))  
        [] xs)
```

Conclusión

`reduce` es la operación base. `map` y `filter` son casos particulares con un acumulador que resulta ser una colección.



3. Profundizando en reduce

Las dos formas de reduce



```
(reduce f coll)           ;; sin valor inicial  
(reduce f init coll)     ;; con valor inicial
```

Sin init, el primer elemento hace de acumulador inicial:

```
(reduce + [1 2 3])        ;; => 6  
(reduce + [42])           ;; => 42  (nunca llama a +)  
(reduce + [])             ;; => 0   (devuelve (+), el neutro)  
(reduce + 100 [1 2 3])    ;; => 106
```

¿Por qué importa el valor inicial?



- Define qué se devuelve cuando la colección está **vacía**.
- Fija el **tipo** del acumulador, que no tiene por qué coincidir con el de los elementos.

Este segundo punto es el que abre la puerta a casi todo: el acumulador puede ser un número, un texto, un vector o un mapa, mientras los elementos son otra cosa.

Ejercicio

Escribir una función *contar-apariciones* que cuente cuántas veces aparece cada palabra. Entran textos, sale un mapa.

```
(def v ["a" "b" "a" "c" "a"])
(contar-apariciones v)
; => {"a" 3, "b" 1, "c" 1}
```


Contemos cuántas veces aparece cada palabra. Entran textos, sale un mapa:

```
(reduce (fn [conteo palabra]
          (update conteo palabra (fnil inc 0)))
        {}
        ["a" "b" "a" "c" "a"])
;; => {"a" 3, "b" 1, "c" 1}
```

- (fnil inc 0) usa inc si la clave existe y arranca en 0 si todavía no.
- El acumulador (un mapa) tiene un tipo distinto a los elementos (textos).

¿Qué pasa si no queremos recorrer todo el arreglo?

```
int sum_until_exceeded(vector<int> &a) {  
    int total = 0;  
    for (int i = 0; i < a.size(); i++) {  
        total += a[i];  
        if (total > 100) break;  
    }  
    return total;  
}
```

Cortar antes de tiempo con `reduced`



`reduced` envuelve el acumulador y detiene la reducción de inmediato:

```
(reduce (fn [acc x]
          (if (> acc 100)
              (reduced acc)
              (+ acc x)))
        0
        (range))
;; => 105
```

- `(range)` es infinita, pero la reducción termina apenas se cumple la condición.
- Es el equivalente funcional del `break` de un bucle imperativo.



4. reductions: conservar la historia



Tomado de: Codeforces Div2 ronda 248

Camila tiene n piedras, numeradas del 1 al n . El costo de la i -ésima piedra es v_i .

Nos hará m preguntas sobre la suma de los costos en ciertos rangos. Hay 2 tipos de preguntas:

Para cada pregunta nos darán un rango desde el índice l hasta el índice r :

1. **Tipo 1: Arreglo Original** Calcular la suma en el arreglo tal como se nos entregó:

$$\sum_{i=l}^r v_i$$

2. **Tipo 2: Arreglo Ordenado** Calcular la suma si las piedras estuvieran ordenadas de menor a mayor $(u_1, u_2, \dots, u_n)$:

$$\sum_{i=l}^r u_i$$



- **Lista v :** lista de enteros que corresponden a los valores individuales.
- **Lista q :** lista de consultas de la forma $[t, l, r]$, donde t es el tipo de consulta y l, r son los límites del rango.

Trabajaremos con un caso pequeño para ver la solución paso a paso:

```
;; TEST 2
(def v [5 5 2 3])
(def q [[1 2 4] [2 1 4] [1 1 1] [2 1 4]
        [2 1 2] [1 1 1] [1 3 3] [1 1 3]
        [1 4 4] [1 2 2]])
```

Cada consulta tiene tres datos: tipo de arreglo, límite izquierdo y límite derecho.

Ejercicio

Proponga una solución al problema



1. ¿Qué es la suma de prefijos? (Prefix sum en inglés)

Idea clave: suma de prefijos



1. ¿Qué es la suma de prefijos? (Prefix sum en inglés)
2. ¿Para qué se usa la suma de prefijos?



1. ¿Qué es la suma de prefijos? (Prefix sum en inglés)
2. ¿Para qué se usa la suma de prefijos?
3. ¿Cómo se implementaría una prefix en clojure?

Para responder rápido una suma de rango, guardamos las sumas acumuladas desde el inicio:

```
;; Agregamos un 0 al inicio para facilitar los rangos
(def pfx_1 (reduce #(conj \%1 (+ (last \%1) \%2)) [0] v))
;; => [0 5 10 12 15]

(def pfx_2 (reduce #(conj \%1 (+ (last \%1) \%2)) [0] (sort
  v)))
;; => [0 2 5 10 15]
```

- pfx_1 sirve para las consultas sobre el arreglo original.
- pfx_2 sirve para las consultas sobre el arreglo ordenado.

Si tenemos las sumas de prefijos, la suma entre l y r se obtiene restando dos posiciones:

```
(defn pfx_query [pfx l r]
  (- (nth pfx r) (nth pfx (dec l))))
```

Por ejemplo, para sumar desde 2 hasta 4 en el arreglo original:

```
(pfx_query pfx_1 2 4)
;; (- (nth pfx_1 4) (nth pfx_1 1))
;; (- 15 5) => 10
```

Pregunta: ¿Qué haría falta para resolver el problema?

Recorremos la lista de consultas con map:

```
(map (fn [[type l r]]  
      (if (= type 1)  
          (pfx_query pfx_1 l r)  
          (pfx_query pfx_2 l r)))  
  q)
```

- Si type es 1, usamos pfx_1.
- Si type es 2, usamos pfx_2.



reduce entrega solo el acumulador final. A veces queremos **todos los estados intermedios**: el saldo después de cada movimiento, el máximo hasta cada punto, el avance acumulado.

Para eso existe `reductions`.

`reductions` hace el mismo recorrido que `reduce`, pero devuelve la secuencia de acumuladores parciales:

```
(reduce + [1 2 3 4])  
;; => 10
```

```
(reductions + [1 2 3 4])  
;; => (1 3 6 10)
```

```
(reductions + 0 [1 2 3 4])  
;; => (0 1 3 6 10)
```

El último elemento de `reductions` es siempre lo que habría devuelto `reduce`.



¿Cómo se podría mejorar la solución al problema usando *reductions*?

¿Qué error hay en la siguiente solución?

```
(def pfx_1 (reductions + v))  
(def pfx_2 (reductions + (sort v)))
```

Característica	reduce	reductions
Devuelve	El acumulador final	Todos los acumuladores
Forma de salida	Un valor	Una secuencia
Evaluación	Estricta	Perezosa
Pregunta que responde	¿Cuál es el total?	¿Cómo evolucionó el total?
Caso típico	Suma, conteo, mapa	Saldo, máximo corrido

Misma maquinaria de recorrido; cambia qué tanto conservamos del camino.



5. Cuando reduce no alcanza: loop/recur

¿Cuál es la diferencia entre estas funciones?

```
(defn factorial-1 [n]
  (if (= n 1)
    1
    (* n (factorial-comun (dec n)))))

(defn factorial-2 [n acc]
  (if (= n 1)
    acc ;
    (factorial-cola (dec n) (* acc n))))
```

Is this assembly?



Presentamos la función loop y recur.

```
(loop [n 5  
      acc 1]  
  (if (zero? n)  
      acc  
      (recur (dec n) (* acc n))))  
;; => 120
```

- loop fija un punto de retorno y los valores iniciales.
- recur vuelve a ese punto con nuevos valores, sin crecer la pila.

Observe que reduce es un loop/recur con un patrón fijo:

```
(defn mi-reduce [f init coll]
  (loop [acc init
        xs coll]
    (if (empty? xs)
        acc
        (recur (f acc (first xs))
                (rest xs)))))
```

reduce fija el recorrido (un elemento tras otro, hasta vaciar la colección). loop/recur nos devuelve el control de ese recorrido cuando lo necesitamos.

¿Cuándo bajar a loop/recur?



La regla práctica: primero map, filter, reduce o reductions. Solo si ninguno encaja, loop/recur. Casos donde sí encaja:

- No iteramos sobre una colección, sino que **generamos un estado** hasta cumplir una condición de parada.
- Avanzamos a **ritmo irregular**: a veces consumimos uno, a veces varios elementos.
- Llevamos **varios acumuladores** a la vez con una lógica de parada propia.

La longitud de la secuencia de Collatz: no recorremos una colección dada, generamos números hasta llegar a 1.

```
(defn pasos-collatz [n]
  (loop [x      n
        pasos 0]
    (if (= x 1)
      pasos
      (recur (if (even? x) (quot x 2) (inc (* 3 x)))
              (inc pasos)))))

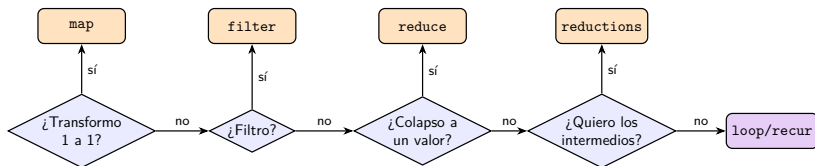
(pasos-collatz 6) ;; => 8
```

No hay colección de entrada que reducir: el siguiente valor depende del actual.



6. ¿Qué herramienta uso?

Un criterio de decisión



Una sola sintaxis ocultaba varias decisiones

El bucle imperativo mezclaba en un mismo `for` la transformación, el filtrado, la acumulación y el control del recorrido. Cada herramienta de hoy aísla una de esas intenciones y le pone nombre.

`loop/recur` queda al final del diagrama a propósito: es la salida cuando el problema ya no tiene la forma de “recorrer una colección”.



7. Ejercicios Prácticos

Ejercicios Prácticos

Reto 0: fila n del Triángulo de Pascal



Implementa el problema del inicio usando una reducción.
Pista: Los coeficientes binomiales cumplen que

$$c_k = \frac{c_{k-1} \cdot (n - k)}{k}$$



```
(defn pascal [n]
  (reductions (fn [c k] (/ (* c (- n k)) k))
    1
    (range 1 n)))

(pascal 5) ;; => (1 4 6 4 1)
```




El siguiente bucle en Java cuenta las vocales de una palabra:

```
int vocales = 0;
for (char c : palabra.toCharArray()) {
    if ("aeiou".indexOf(c) >= 0) {
        vocales++;
    }
}
```

Objetivo: expresarlo en Clojure con `reduce`, identificando primero el valor inicial y la función de combinación.

El valor inicial es 0 y la combinación suma 1 cuando el carácter es vocal:

```
(defn vowel? [letra]
  (contains? (set "aeiou") letra))

(defn cuenta-vocales [palabra]
  (reduce
    (fn [acc x]
      (if (vowel? x) (inc acc) acc))
    0
    palabra))

(cuenta-vocales "programacion") ;; => 5
```

Reto 2: construir un mapa con reduce



Dada una colección de votos:

```
(def votos [:si :no :si :si :no :abstencion])
```

Objetivo: usar reduce para devolver un mapa con el conteo de cada opción.



Mismo patrón que el conteo de palabras, ahora sobre claves:

```
(reduce (fn [conteo voto]
          (update conteo voto (fn nil inc 0)))
        {}
        votos)
;; => {:si 3, :no 2, :abstencion 1}
```

Con el saldo inicial y los movimientos:

```
(def saldo-inicial 1000)
(def movimientos [-300 -400 -500 200])
```

Objetivo:

1. Calcular el saldo después de cada movimiento con reductions.
2. Encontrar el primer saldo que queda en negativo.

`reductions` reconstruye la historia del saldo; `filter` con `first` encuentra el primer negativo:

```
(def saldos
  (reductions + saldo-inicial movimientos))
;; => (1000 700 300 -200 0)

(first (filter neg? saldos))
;; => -200
```

Gracias a la pereza, `filter` detiene el recorrido en cuanto encuentra el primer saldo negativo: no calcula de más.

Reto 4: cuando solo queda loop/recur



El máximo común divisor por el algoritmo de Euclides: mientras $b \neq 0$, se reemplaza el par (a, b) por $(b, a \bmod b)$.

Objetivo: implementar $(\text{mcd } a \text{ } b)$ con loop/recur.

Pregunta para discutir: ¿por qué este problema no encaja cómodamente en reduce?

```
(defn mcd [a b]
  (loop [a a
        b b]
    (if (zero? b)
      a
      (recur b (rem a b)))))
```

```
(mcd 48 36) ;; => 12
```

- No hay colección de entrada: el siguiente par $(b, a \bmod b)$ se calcula a partir del actual.
- El número de pasos no se conoce de antemano; depende de los valores, no de un tamaño.

- El bucle imperativo que acumula un resultado es una reducción.
- `reduce` colapsa; `reductions` conserva el camino; ambos comparten la misma maquinaria de recorrido.
- `loop/recur` es la herramienta de bajo nivel, reservada para cuando la forma del problema no es recorrer una colección.

¡Gracias!